ENTITY FRAMEWORK CORE CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & DBCONTEXT

Installation

```
dotnet add package Microsoft.EntityFrameworkC...
dotnet add package Microsoft.EntityFrameworkC...
public class AppDbContext : DbContext {
public DbSet<User> Users { get; set; }
public DbSet<Order> Orders { get; set; }
protected override void OnConfiguring(DbConte...
=> o.UseSqlServer(connectionString);
}
```

02. ENTITY CONFIG

Architecture

```
// Data Annotations:
[Table("users")] [Key] [Required]
[MaxLength(100)] [Column("full_name")]
// Fluent API (in OnModelCreating):
modelBuilder.Entity<User>().HasKey(u => u.Id);
modelBuilder.Entity<User>().Property(u => u.E...
.IsRequired().HasMaxLength(200).IsUnicode(fal...
```

03. CRUD OPERATIONS

YAML / Config

```
// Create:
db.Users.Add(new User { Name = "Alice" });
await db.SaveChangesAsync();
// Read:
var user = await db.Users.FindAsync(id);
var users = await db.Users.Where(u => u.Activ...
// Update: load  modify  SaveChanges()
// Delete: Remove(entity)  SaveChanges()
```

04. MIGRATIONS

Workflow

```
dotnet ef migrations add InitialCreate
dotnet ef database update
dotnet ef migrations remove  (last uncommitte...
dotnet ef database update 20240101_AddEmail  ...
dotnet ef database update 0  (rollback all)
dotnet ef migrations script  (generate SQL)
```

05. LINQ QUERIES

CRUD API

```
db.Users.Where(u => u.Role == "Admin")
.OrderBy(u => u.Name)
.Select(u => new { u.Id, u.Name })
.Skip(20).Take(10)
.ToListAsync()
db.Users.AnyAsync(u => u.Email == email)
db.Users.CountAsync(u => u.Active)
```

06. RELATIONSHIPS

Integration

```
// One-to-many:
public class User { public ICollection<Order>...
public class Order { public int UserId { get;...
public User User { get; set; } }
// Config:
modelBuilder.Entity<Order>().HasOne(o => o.Us...
.WithMany(u => u.Orders).HasForeignKey(o => o...
```

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. EAGER/LAZY/EXPLICIT LOADING

Hardening

```
// Eager:
db.Orders.Include(o => o.User).Include(o => o...
// Lazy (requires proxies + virtual nav props...
var user = await db.Users.FindAsync(id);
var orders = user.Orders; // lazy loaded
// Explicit:
await db.Entry(user).Collection(u => u.Orders...
```

08. TRACKING vs NO-TRACKING

Observability

```
// Tracked (default)  change detection runs o...
var user = await db.Users.FindAsync(id);
user.Name = "New"; await db.SaveChangesAsync(...
// No-tracking (read-only, faster):
db.Users.AsNoTracking().Where(u => u.Active)....
```

Use AsNoTracking() for pure read operations

09. TRANSACTIONS

Optimization

```
using var tx = await db.Database.BeginTransac...
try {
await db.SaveChangesAsync();
await db.SaveChangesAsync(); // second operat...
await tx.CommitAsync();
} catch { await tx.RollbackAsync(); throw; }
// SaveChanges is a single implicit transacti...
```

10. RAW SQL

Unit Tests

```
var users = db.Users.FromSqlRaw(
"SELECT * FROM users WHERE status = {0}", sta...
db.Database.ExecuteSqlRaw(
"UPDATE users SET active = 1 WHERE id = {0}",...
// Interpolated (parameterized automatically):
db.Users.FromSqlInterpolated($"SELECT * FROM ...
```

11. COMMON ANNOTATIONS

Commands

```
[Key] [DatabaseGenerated(DatabaseGeneratedOpt...
[Required] [MaxLength(200)] [Column("col_name...
[Table("table_name")] [Index(nameof(Email), I...
[NotMapped]  // exclude property from DB
[Timestamp]  // optimistic concurrency token
[ForeignKey(nameof(UserId))]
```

12. COMMON GOTCHAS

Warning

DbContext is not thread-safe one per request (Scoped)
N+1 queries always Include() navigation properties
Mixing tracked and untracked entities causes conflicts
Cascade delete enabled by default for required relationships
Complex queries: check generated SQL with .ToQueryString()